CSED233 · 데이터구조 · 포스텍

정리본 8. Sorting

단순 정렬 vs 고급 정렬 · 각 알고리즘의 동작 · 언제 무엇을 고를까.

1. 정렬이란 + 알고리즘 지도

Sorting(정렬): (key, info) 쌍으로 이루어진 원소 컬렉션을 key 기준으로 재배열하는 것.

단순 정렬 · $O(n^2)$

- Bubble Sort(거품 정렬)
- Selection Sort(선택 정렬)
- Insertion Sort(삽입 정렬)

구현이 간단, 작은 데이터용.

고급 정렬 · O(n log n)

- Merge Sort(병합 정렬)
- Quick Sort(퀵 정렬)
- Heap Sort(힙 정렬)

구현은 복잡하지만 실무 표준. 단, 고급 정렬을 모두 O(n log n)이라고 분류하는 것은 평균적인 성능을 설명할 때만 적절하다 — 최악 시간복잡도까지 O(n log n)이 보장되는 것은 Merge Sort와 Heap Sort이며, 일반적인 Quick Sort의 최악 시간복잡도는 $O(n^2)$이다.

특수 정렬

- Counting Sort(계수 정렬)
- Radix Sort(기수 정렬)

특정 데이터 타입에서만 가능.

2. 단순 정렬 ($O(n^2)$)

2.1 Bubble Sort(버블 정렬)

인접한 두 원소를 비교·교환하며 가장 큰 값을 맨 뒤로 떠오르게 한다. 한 Pass마다 맨 뒤 한 칸이 확정.

예제 · [5, 3, 8, 1, 2]

```
Pass 1: (5,3)→교환 [3,5,8,1,2] · (5,8)→유지 · (8,1)→교환 · (8,2)→교환
      [3, 5, 1, 2, |8|]  ← 8 확정
Pass 2: (3,5)→유지 · (5,1)→교환 · (5,2)→교환
      [3, 1, 2, |5, 8|]  ← 5 확정
Pass 3: (3,1)→교환 · (3,2)→교환
      [1, 2, |3, 5, 8|]  완성
```

시간: 최선 $O(n)$ (교환이 없으면 조기 종료하는 최적화 적용 시), 평균/최악 $O(n^2)$ · 공간 $O(1)$ · Stable ✓

2.2 Selection Sort(선택 정렬)

정렬 안 된 부분에서 최솟값을 찾아 맨 앞과 교환한다. 한 Pass마다 맨 앞 한 칸이 확정.

예제 · [5, 3, 8, 1, 2]

```
Pass 1: 최솟값 1 → 맨 앞과 swap → [1, 3, 8, 5, 2]
Pass 2: 최솟값 2 → 두 번째와 swap → [1, 2, 8, 5, 3]
Pass 3~: [1, 2, 3, 5, 8] 완성
```

시간: 항상 $O(n^2)$ · 공간 $O(1)$ · **Unstable ✗** · swap은 최대 n−1회(Bubble보다 적음)

2.3 Insertion Sort(삽입 정렬)

정렬된 앞부분을 유지하며, 다음 원소를 알맞은 위치에 삽입한다.

예제 · [5, 3, 8, 1, 2]

```
[5 | 3, 8, 1, 2]   3 삽입 → [3, 5 | 8, 1, 2]
[3, 5 | 8, 1, 2]   8 삽입 → [3, 5, 8 | 1, 2]
…                  → [1, 2, 3, 5, 8] 완성
```

거의 정렬된 데이터

Insertion Sort는 **거의 정렬된 입력에서** $O(n)$에 가깝게 동작한다 — 각 원소가 거의 이동하지 않기 때문. 실무 라이브러리가 작은 구간에서 삽입 정렬로 전환하는 이유.

시간: 최선 $O(n)$, 평균/최악 $O(n^2)$ · 공간 $O(1)$ · Stable ✓

3. 고급 정렬 ($O(n \log n)$)

3.1 Merge Sort(병합 정렬) — 분할 정복

① **분할**: 배열을 반씩 계속 쪼갠다 — 깊이 $O(\log n)$.

② **병합**: 두 정렬된 조각을 합치며 정렬 — 각 층 $O(n)$.

예제 · [5, 3, 8, 1, 2, 9]

```
분할: [5,3,8] [1,2,9] → [5,3][8] [1,2][9] → [3][5] ⋯
병합: [3,5] + [8] → [3,5,8] · [1,2] + [9] → [1,2,9]
    [3,5,8] + [1,2,9] → [1, 2, 3, 5, 8, 9]
```

시간: **항상 $O(n \log n)$ — 최악에도 보장** · 공간 $O(n)$(임시 배열) · Stable ✓ · 실행시간 예측이 중요한 환경에 적합

Non-recursive Merge Sort (Bottom-up)

재귀 분할 없이, 배열을 이미 쪼개져 있다고 보고 **width = 1, 2, 4, ⋯** 단위로 이웃 조각을 병합해 올라간다. 각 단계 $O(n)$, 단계 수 $O(\log n)$ → 총 $O(n \log n)$. 재귀 호출 오버헤드와 stack overflow 위험이 없어 대용량에 유리.

예제 · [5, 3, 8, 1, 2, 9]

```
width 1: [5][3] [8][1] [2][9] → [3,5] [1,8] [2,9]
width 2: [3,5] [1,8] → [1,3,5,8] · [2,9]
width 4: [1,3,5,8] + [2,9] → [1, 2, 3, 5, 8, 9]
```

3.2 Quick Sort(퀵 정렬) — Pivot 기반 분할

① **Pivot 선택**(예: 맨 왼쪽 또는 중간값).

② **분할**: pivot 미만 | pivot | pivot 이상으로 나눈다 (같은 값은 오른쪽으로) — $O(n)$.

③ 양쪽을 **재귀** 정렬. pivot이 잘 갈라지면 깊이 $O(\log n)$.

예제 · [5, 3, 8, 1, 2], pivot = 5

```
<5: [3,1,2]  =5: [5]  >5: [8]
재귀: [3,1,2] → [1,2,3]
최종: [1, 2, 3, 5, 8]
```

시간: 평균 $O(n \log n)$, **최악 $O(n^2)$(pivot 선택 실패 시)** · 공간 평균 $O(\log n)$, 최악 O(n)(재귀 스택) · Unstable ✗

배열 기반 in-place Partition을 사용하는 Quick Sort는 인접한 메모리 영역을 효율적으로 접근하고 추가 데이터 저장 공간이 적어, 실제 환경에서 좋은 캐시 효율과 높은 실행 성능을 보이는 경우가 많다. 그러나 Quick Sort가 항상 가장 빠른 정렬 알고리즘인 것은 아니다 — 실제 실행 속도는 데이터 크기와 분포, 중복 키의 비율, Pivot 선택 방식, Partition 구현, 캐시 구조, 비교 비용 등에 영향을 받는다. 따라서 Quick Sort는 평균적으로 효율적이고 실제 구현에서 빠른 경우가 많은 알고리즘이라고 설명하는 것이 적절하며, 모든 상황에서 가장 빠르다고 단정해서는 안 된다.

In-place Partitioning (Two-Pointer Scan)

양 끝 포인터 L, R이 서로를 향해 이동하며 교환. L은 pivot 이상인 값, R은 pivot 미만인 값을 찾아 멈춘다.

예제 · [17, 20, 17, 33, 15, 2, 21, 14], pivot = 17

```
L: 20(≥17) 발견, R: 14(<17) 발견 → swap
 [17, 14, 17, 33, 15, 2, 21, 20]
L: 17 발견, R: 2 발견 (21은 ≥17이라 스킵) → swap
 [17, 14, 2, 33, 15, 17, 21, 20]
L: 33 발견, R: 15 발견 → swap
 [17, 14, 2, 15, 33, 17, 21, 20]
L과 R이 엇갈림 → pivot을 R 위치(15)와 교환
 [15, 14, 2] | 17 | [33, 17, 21, 20]
→ 좌우를 각각 재귀 → 최종 [2, 14, 15, 17, 17, 20, 21, 33]
```

엇갈린 뒤의 마지막 교환

스캔이 끝나면 pivot은 **R이 가리키는 위치**와 교환한다(L이 아니라 R). 그래야 pivot 왼쪽이 모두 pivot 이하가 된다.

3.3 Heap Sort(힙 정렬)

배열을 Max-Heap으로 만든 다음, 루트에 있는 최댓값을 배열의 맨 뒤로 이동시키는 과정을 반복하는 정렬이다. Max-Heap에서는 모든 부모 노드의 키가 자식 노드의 키보다 크거나 같으므로, 루트에는 현재 힙의 최댓값이 위치한다.

① Heapify — Max-Heap 구성: 배열의 마지막 내부 노드부터 루트 방향으로 Down-heap을 수행하는 Bottom-up Heapify로 O(n) 시간에 구성한다 (정리본 7).

② 루트와 마지막 원소 교환: 힙의 루트 A[1]에 있는 현재 최댓값을 현재 힙의 마지막 원소 A[heapSize]와 교환한다. 그러면 최댓값이 배열의 정렬 완료 구간 맨 앞에 놓인다.

③ 힙 크기 감소: heapSize를 1 줄여 방금 확정한 최댓값을 힙에서 제외한다. 배열에서 실제 원소를 삭제하는 것이 아니라, 힙으로 취급하는 구간만 줄인다.

④ Down-heap으로 힙 복구: 루트로 이동한 원소가 Max-Heap 조건을 위반할 수 있으므로, 더 큰 자식과 교환하면서 아래로 내려보내 Max-Heap 속성을 복구한다. 이 과정은 O(log n) 시간이 걸린다.

⑤ 반복: 힙 크기가 1이 될 때까지 2~4단계를 반복한다. 매 반복마다 최댓값이 뒤에서부터 확정되므로 최종적으로 배열 전체가 오름차순 정렬된다.

예제 · [8, 3, 5, 1, 2] (| 오른쪽은 정렬 완료 구간)

초기 Max-Heap: [8, 3, 5, 1, 2] — 이미 Max-Heap 조건을 만족해 Heapify 후에도 그대로
Pass 1: 루트 8 ↔ 마지막 2 교환 [2, 3, 5, 1 | 8] → heapSize 5→4 → Down-heap(2↔5) [5, 3, 2, 1 | 8]
Pass 2: 루트 5 ↔ 마지막 1 교환 [1, 3, 2 | 5, 8] → heapSize 4→3 → Down-heap(1↔3) [3, 1, 2 | 5, 8]
Pass 3: 루트 3 ↔ 마지막 2 교환 [2, 1 | 3, 5, 8] → heapSize 3→2 → 루트 2가 자식 1보다 커 추가 교환 없음
Pass 4: 루트 2 ↔ 마지막 1 교환 [1 | 2, 3, 5, 8] → heapSize 2→1 → 정렬 완료 [1, 2, 3, 5, 8]

시간은 최선/평균/최악 모두 O(n log n) (Heapify O(n) + 추출당 Down-heap O(log n)), 공간 O(1) (in-place, 반복식 Down-heap 기준), Unstable ✗. 입력 순서에 관계없이 O(n log n)의 최악 시간복잡도를 보장하고, 추가 배열 없이 제자리(in-place) 정렬이 가능하다.

4. 정렬 비교표

알고리즘	최선	평균	최악	공간	안정성
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗

5. 어떤 정렬을 고를까

데이터 크기별

- 작은 데이터 (예: 수십 개의 원소) → **Insertion**: 구현이 간단하고 재귀 호출이나 추가 자료구조를 위한 오버헤드가 작다. 특히 이미 정렬되었거나 거의 정렬된 경우 효율적이다.
- 중간 크기 데이터 → **Quick**: 평균 O(n log n)이고 배열 기반 구현에서 메모리 접근 효율이 높은 경우가 많다. 다만 Pivot 선택과 입력 분포에 따라 성능이 달라진다.
- 큰 데이터 → **Merge / Quick / Heap** 모두 고려: Merge는 최악 O(n log n)과 안정성이 필요한 경우에, Heap은 추가 메모리가 매우 제한적일 때 유리하고, Quick도 적절히 구현하면 대용량 데이터에서 효율적일 수 있다.

데이터 특성별

- 거의 정렬된 데이터 → **Insertion**: 역전 쌍 개수를 I라고 하면 실행 시간은 O(n + I)이므로, I가 작을수록 효율적이다.
- 안정성 필수(다중 키 정렬 등) → **Merge / Insertion**: Stable Sort는 같은 키를 가진 원소들 사이의 상대 순서를 보존한다. Bubble도 안정적으로 구현할 수 있지만 큰 데이터에서는 일반적으로 비효율적이다.
- 추가 메모리 제약 → **Heap** 우선 고려: 반복적 Down-heap을 사용하는 in-place Heap Sort는 O(1)의 추가 공간만 필요하다. 일반적인 in-place Quick Sort는 별도의 병합 배열은 필요하지 않지만 재귀 스택으로 평균 O(log n), 최악 O(n)의 추가 공간을 사용하므로, Quick Sort를 추가 메모리 O(1)인 알고리즘으로 분류해서는 안 된다. 일반적인 배열 기반 Merge Sort는 병합용으로 O(n)의 추가 공간이 필요하다.
- 최악 시간복잡도 보장 필요 (실시간, 금융) → **Merge / Heap**: 두 알고리즘 모두 최악 O(n log n)을 보장한다. 반면 일반적인 Quick Sort는 최악 $O(n^2)$이므로, 최악 성능이 중요한 경우에는 Heap Sort로 전환하는 IntroSort(Introspective Sort)와 같은 하이브리드 방식도 고려할 수 있다.

위 기준은 일반적인 선택을 돕는 경험칙(heuristic)이며, 모든 환경에 적용되는 절대적인 규칙은 아니다. 실제 성능은 데이터 분포, 하드웨어, 라이브러리 구현, 비교 연산 비용 등에 따라 달라진다. $n \leq 50$이면 Insertion Sort, $50 < n \leq 1{,}000{,}000$이면 Quick Sort, $n > 1{,}000{,}000$이면 Merge Sort와 같은 구간은 보편적인 성능 경계가 아니다 — 이러한 숫자는 특정 구현과 실행 환경에서만 의미가 있을 수 있으며, 실제 전환 기준은 측정 결과에 따라 결정해야 한다.

결론: 정렬 알고리즘 선택에는 데이터 크기만으로 결정되는 보편적인 경계가 없다. 작은 구간이나 거의 정렬된 입력에서는 Insertion Sort, 평균적인 실행 효율이 중요할 때는 Quick Sort, 안정성과 최악 시간 보장이 필요할 때는 Merge Sort, 추가 메모리를 최소화하면서 최악 O(n log n)을 보장해야 할 때는 Heap Sort를 우선 고려할 수 있다.

6. Quiz

Q1. 거의 정렬된 배열에서 가장 빠른 정렬은?

> **힌트**

Insertion Sort에서 원소 하나를 삽입할 때 몇 번의 비교와 이동이 발생하는지 생각해 보자. 특히 배열의 역전 쌍(inversion), 즉 인덱스 $i < j$이면서 $A[i] > A[j]$인 원소 쌍의 개수가 적으면 전체 이동 횟수가 어떻게 달라지는지 살펴보자. 단순히 최선 시간복잡도가 O(n)인 알고리즘을 찾는 것만으로는 충분하지 않다.

> **정답**

개념은 힌트 참고.

정답: Insertion Sort가 일반적으로 유리하다. Insertion Sort는 왼쪽의 정렬된 구간에 다음 원소를 삽입하면서 자신보다 큰 원소들을 오른쪽으로 한 칸씩 이동시키므로, 입력이 거의 정렬되어 있으면 각 원소가 이동해야 하는 거리가 짧고 전체 비교·이동 횟수도 적다. 배열의 역전 쌍 개수를 I라고 하면 표준 Insertion Sort의 실행 시간은 O(n + I)이다 (이미 정렬된 배열에서는 I = 0이라 O(n)). 단, 최선 시간복잡도가 O(n)이라는 사실만으로 Insertion Sort가 유일한 정답이 되는 것은 아니다 — 조기 종료를 적용한 Bubble Sort도 이미 정렬된 입력에서 O(n)이 가능하다. 핵심 장점은 역전 쌍이 적을 때 실제 이동량도 적어진다는 점이다. 일반적인 Merge Sort와 Heap Sort는 입력이 거의 정렬되어 있어도 O(n log n)에 동작하고, Quick Sort는 평균 O(n log n)이지만 최악 O(n^2)이다. 따라서 거의 정렬된 배열에서는 Insertion Sort가 유리한 경우가 많지만, 모든 입력과 구현에서 반드시 가장 빠르다고 단정할 수는 없다.

Q2. Stable sort가 필요한 이유는?

힌트

키가 같은 원소들의 상대 순서가 유지되는가.

정답

개념은 힌트 참고.

정답: Stable Sort(안정 정렬)는 정렬에 사용하는 키가 같은 두 원소의 상대적인 순서를 정렬 후에도 유지하는 알고리즘이다. 즉, Stable Sort는 현재 정렬 키가 같은 원소들 사이에서만 이전 순서를 보존하며, 키가 다른 원소들 사이의 순서까지 보존하지는 않는다. 다중 키 정렬에서는 일반적으로 우선순위가 낮은 키부터 높은 키 순서로 Stable Sort를 적용한다 — 예를 들어 급여를 보조 키, 직급을 주 키로 사용하려면 급여 기준으로 먼저 정렬하고, 이후 직급 기준으로 Stable Sort를 적용하면 직급별로 묶이면서 같은 직급 안에서는 급여순이 유지된다.

Q3. Merge Sort가 Quick Sort보다 느린 편인데 왜 쓸까?

힌트

비교표의 "최악" 열: Quick은 $O(n^2)$, Merge는 $O(n \log n)$.

정답

개념은 힌트 참고.

정답: Merge Sort는 최악에도 O(n log n)의 시간복잡도를 보장하므로 실행 시간의 상한이 중요한 환경에서 유리하다. 일반적인 Quick Sort는 평균 O(n log n)으로 효율적이지만, 불균형한 Partition이 반복되면 최악 $O(n^2)$이 될 수 있다. 따라서 최악 성능 보장이 중요한 상황에서는 Merge Sort가 좋은 선택이 될 수 있다. 다만 실시간 시스템에 적합한지는 실제 최악 실행 시간과 마감 시간 조건까지 고려해야 한다.